

Bug Prediction-ISW2

ISW2

ENRICO BARBATANO

Sommario

1. Introduzione	1
2. Metodologia	2
2.1 Milestone 1 (Dataset Creation).....	2
2.2 Milestone 2 (Classifiers).....	3
2.3 Milestone 3 (What-if Analysis).....	4
2.4 Milestone 4 (Automated Refactoring)	6
3. Risultati	6
3.1 Risultati Milestone 1	6
3.2 Risultati Milestone 2	7
3.3 Risultati Milestone 3	8
3.4 Risultati Milestone 4	8
4. Minacce alla validità e scelte soggettive	10
5. Conclusioni	11
Appendice C	12

1. Introduzione

L'obiettivo del presente studio è analizzare la relazione esistente tra la presenza di **code smell** e la **difettosità del software** all'interno del progetto open-source **Apache Storm** (scelto tramite l'algoritmo indicatoci). L'attività è stata strutturata in quattro fasi principali (milestone):

1. **Creazione del Dataset:** Estrazione di metriche software e identificazione delle classi affette da bug tramite algoritmi di labeling.
2. **Valutazione dei Classificatori:** Identificazione del modello di Machine Learning più accurato per predire la difettosità del sistema.
3. **What-if Analysis:** Studio controfattuale per stimare l'impatto della rimozione totale degli smell sulla riduzione dei bug predetti.
4. **Refactoring Automatizzato:** Valutazione dell'efficacia dell'IA (Microsoft Copilot) nel migliorare la manutenibilità del codice senza alterarne la stabilità.

2. Metodologia

2.1 Milestone 1 (Dataset Creation)

Release analizzate

Per la costruzione del dataset, è stata analizzata la storia evolutiva di Storm, decidendo di **ignorare l'ultimo 66% delle release**. Questa scelta non è stata motivata dalla necessità di mitigare il fenomeno dello **Snoring**, se includessimo le release più recenti, molte classi verrebbero etichettate erroneamente come "sane" solo perché i bug in esse contenuti non sono ancora emersi ("**sleeping defects**"), portando a un alto numero di **falsi negativi** che degraderebbero la qualità dell'addestramento dei modelli.

Selezione delle metriche

Per caratterizzare ciascuna classe sono state selezionate oltre venti metriche software, scelte in modo da descrivere non solo la struttura del codice, ma anche la sua evoluzione nel tempo e il contesto collaborativo in cui viene modificato. La selezione è stata guidata dall'esigenza di utilizzare feature **quantificabili**, **ripetibili** e sufficientemente **robuste rispetto al contesto**, così da fornire un segnale affidabile ai modelli di classificazione.

Nel complesso, le metriche adottate coprono cinque dimensioni complementari del rischio software: **dimensione e complessità strutturale**, **instabilità evolutiva**, **storia dei difetti**, **fattori temporali** e **fattori organizzativi/collaborativi**. In questo modo è possibile cogliere sia la propensione intrinseca della classe a essere difficile da comprendere e mantenere, sia la probabilità che il suo processo evolutivo abbia introdotto condizioni favorevoli alla comparsa di difetti.

L'elenco completo delle metriche considerate e della loro funzione è riportato in [Appendice C](#).

Identificazione dei Code Smell

Per ogni classe Java presente nelle release selezionate di Apache Storm, è stata eseguita un'analisi statica per quantificare il debito tecnico.

- **Strumentazione:** L'attività è stata condotta principalmente tramite **SonarCloud** (o strumenti analoghi come PMD), che permette di scansionare il codice sorgente alla ricerca di violazioni di regole di programmazione, manutenibilità e design.
- **Metrica Nsmells:** Il risultato di questa analisi è la metrica **Nsmells**, che rappresenta il conteggio totale dei difetti testuali rilevati. Nel dataset finale, questa variabile funge da indicatore del debito tecnico accumulato, permettendo di valutare se classi con un alto numero di smell siano effettivamente più soggette a bug.

Labeling

La metodologia di **labeling** utilizzata per il progetto **Apache Storm** ha avuto l'obiettivo di assegnare l'etichetta buggy (YES/NO) a ogni coppia classe-release, integrando l'algoritmo **SZZ** con la tecnica **Proportion Total** per garantire la massima precisione e tracciabilità.

Di seguito viene descritto il processo analitico seguito:

Il labeling ha assegnato l'etichetta buggy (YES/NO) a ogni coppia classe-release seguendo una gerarchia di priorità basata sull'affidabilità dei dati.

Determinazione della Injected Version (IV)

Per stabilire quando un bug è stato introdotto, si è proceduto come segue:

Dati JIRA: Utilizzo delle **Affected Versions** se presenti e consistenti.

Algoritmo SZZ: Identificazione del *Bug-Introducing Change* (BIC) tramite git blame sui commit di fix.

Proportion Total (Fallback): In assenza di informazioni dai primi due metodi, la IV è stata stimata statisticamente.

Calcolo del parametro P tramite Media

Per applicare la tecnica Proportion, è stato calcolato il parametro analizzando i ticket "puliti" (quelli con IV, OV e FV certe). Dalle analisi sono emersi due valori rappresentativi:

Mediana: 2.0

Media: circa 2.76

Per la configurazione finale del progetto Storm, si è deciso di utilizzare la **media**. Questa scelta permette di riflettere il valore atteso della distribuzione del ciclo di vita dei bug all'interno del progetto, bilanciando la stima della release di introduzione.

Applicazione della stima

La versione di introduzione mancante è stata quindi calcolata con la formula:

$$\text{Predicted IV} = \text{FV} - (\text{FV} - \text{OV}) * 2.76$$

Regola di Labeling Finale

Una volta determinata la IV (reale o stimata), ogni classe coinvolta nel bug è stata marcata come buggy = YES per tutte le release tali che:

$$IV \leq r < OV$$

Questo garantisce che il dataset rifletta accuratamente lo stato difettoso del software nel periodo intercorso tra la sua introduzione "silenziosa" e la segnalazione ufficiale del ticket.

2.2 Milestone 2 (Classifiers)

L'obiettivo di questa fase è stato identificare il modello di Machine Learning più efficace nel predire se una classe di **Apache Storm** sia affetta da bug, basandosi sulle metriche estratte precedentemente.

Preprocessing dei Dati

Prima dell'addestramento, il dataset è stato sottoposto a una pipeline di preparazione in **WEKA**:

- **Rimozione Metadati:** Sono stati eliminati gli attributi identificativi (project, release, nameclass) per evitare che il modello apprendesse pattern basati su nomi o identificativi univoci invece che su proprietà strutturali del codice.
- **Normalizzazione:** Poiché le metriche presentano scale eterogenee (es. le LOC possono essere migliaia, mentre la bug_density è decimale), è stata applicata la normalizzazione per rendere le feature omogenee. Questo passaggio è cruciale per algoritmi basati sulla distanza come **IBk**.

Feature Selection (Approccio Filter)

In conformità con i requisiti, è stata applicata una selezione degli attributi di tipo **Filter**, che valuta le feature indipendentemente dal classificatore.

- **Strumentazione:** È stato utilizzato l'evaluator **CfsSubsetEval** (che premia sottoinsiemi di feature correlate con la classe target ma poco ridondanti tra loro) insieme al metodo di ricerca **BestFirst** in modalità *forward search* con backtracking.
- **Risultato:** Sono state selezionate 7 feature chiave: `loc`, `loc_added`, `fan_out`, `revisions_density`, `age`, `bug_density` e, mantenuta manualmente, `code_smells`.

Gestione dello Sbilanciamento (Balancing)

Il dataset originale presentava uno sbilanciamento critico: le classi *buggy* (YES) rappresentano solo il **7%** del totale.

Per evitare che i classificatori ignorassero la classe minoritaria (problema evidenziato dalla baseline **ZeroR**, che ottiene alta accuratezza nominale ma Recall pari a zero per i bug), è stata applicata la tecnica **SMOTE**.

- **Prevenzione del Data Leakage:** SMOTE è stato configurato all'interno di un **FilteredClassifier**. In questo modo, le istanze sintetiche vengono generate solo sulla porzione di *training* di ogni fold, garantendo che i dati di *test* rimangano reali e non contaminati.

Validazione e Classificatori

L'accuratezza è stata misurata confrontando tre algoritmi: **RandomForest** (ensemble di alberi), **NaiveBayes** (probabilistico) e **IBk** (lazy/vicinato).

- **Tecnica di Validazione:** È stata adottata la 10 times 10-fold cross-validation, eseguendo quindi 100 valutazioni indipendenti per ogni configurazione per garantire la stabilità statistica dei risultati.
- **Metriche di Valutazione:** La scelta del miglior modello si è basata su Area under ROC (**AUC**) e **Kappa Statistic**
 - La **Kappa Statistic** è stata dunque selezionata per misurare quanto il modello sia effettivamente più accurato rispetto a un classificatore casuale, penalizzando i modelli che ottengono buoni risultati basandosi solo sulla classe maggioritaria.
 - L'**Area Under ROC (AUC)** è stata considerata una metrica centrale poiché misura la capacità del modello di distinguere tra classi sane e difettose indipendentemente dalla soglia decisionale impostata, fornendo una valutazione della robustezza del classificatore superiore a quella delle metriche dipendenti dalla soglia.

2.3 Milestone 3 (What-if Analysis)

L'obiettivo di questa fase è stato stimare quante classi buggy avrebbero potuto essere evitate se il progetto fosse stato privo di code smell, mantenendo però invariate tutte le altre caratteristiche del software e del processo di sviluppo.

Costruzione dei Dataset

Seguendo rigorosamente la procedura definita nel paper di riferimento, sono stati strutturati quattro dataset:

- **Dataset A:** Il dataset originale completo derivante dalla Milestone 1 (9864 istanze).

- **Dataset B+**: Il sottoinsieme di A che include esclusivamente le classi affette da almeno un code smell, corrispondente a 6436 istanze.
- **Dataset C**: Il sottoinsieme di A contenente le classi già prive di smell (code_smell=0), pari a 3428 istanze.
- **Dataset B (Scenario Controfattuale)**: Questo dataset sintetico è stato generato partendo da B+ e impostando artificialmente la feature code_smells a **0** per tutte le istanze, lasciando inalterate le altre metriche (LOC, Churn, Age, ecc.) per isolare l'effetto del debito tecnico.

Procedura di Valutazione e Scelta del Predittore

Come strumento di predizione (**BClassifier**) è stato selezionato il modello risultato più accurato nella Milestone 2.

Il modello è stato addestrato sull'intero **Dataset A** per catturare le relazioni (spesso non lineari) tra smell e bug. Successivamente, il classificatore così addestrato è stato utilizzato per predire la difettosità sui dataset A, B+, B e C. Per rendere questa valutazione completa è stato definito un confronto basato sull'incrocio tra dati reali e predizioni del modello sui diversi scenari.

Struttura del Confronto

L'analisi si basa sul monitoraggio di due metriche fondamentali applicate ai quattro dataset (A, B+, B, C):

- **Actual Buggy (A)**: Il numero reale di classi etichettate come difettose nel dataset originale tramite il labeling SZZ/Proportion.
- **Estimated Buggy (E)**: Il numero di classi che il **BClassifier** predice come difettose.

Il confronto incrociato tra questi valori risponde a precise necessità metodologiche:

1. **Validazione dell'Oracolo (Dataset A)**: Il confronto tra *Actual* ed *Estimated* sul set completo originale (A) serve a dimostrare che il classificatore è un "oracolo" affidabile.
2. **Isolamento della Variabile "Debito Tecnico" (B+ vs B)**: Questo è il cuore dell'analisi controfattuale. Confrontando le classi predette difettose in **B+** (classi "smelly" reali) con quelle in **B** (le stesse classi con smell azzerati artificialmente), è possibile isolare l'impatto specifico degli smell. Poiché tutte le altre metriche (LOC, Churn, Age) rimangono invariati, ogni variazione nell'esito *Estimated* è attribuibile esclusivamente all'influenza dei code smell sulle decisioni del modello.
3. **Stima delle Classi Prevenute**: Il valore fondamentale cercato è la differenza $\Delta Buggy = E(B+) - E(B)$. Questo dato rappresenta la stima di quante classi, attualmente considerate a rischio dal modello, risulterebbero "sane" in uno scenario a debito tecnico zero.
4. **Baseline di Controllo (Dataset C)**: L'inclusione del dataset C permette di osservare come il modello si comporta su classi che, nel progetto reale, erano già prive di smell. Questo serve a verificare se la logica del classificatore sulle classi "pulite" naturali (C) sia coerente con quella applicata alle classi "pulite" artificialmente (B).

2.4 Milestone 4 (Automated Refactoring)

L'obiettivo di quest'ultima fase è stato valutare l'efficacia delle tecnologie di **Generative IA** (nello specifico Microsoft Copilot) nell'eseguire attività di refactoring su classi reali di **Apache Storm**, misurando la capacità dello strumento di ridurre il debito tecnico senza introdurre regressioni funzionali.

Le classi selezionate secondo l'algoritmo fornito, e con una scrematura (sono state eliminate quelle con LOC<70 e numero di metodi<5) sono risultate:

- **StatsUtil.java** (Caso complesso): 265 smell, 1770 LOC. Rappresenta una "God Class" con logiche di aggregazione critiche.
- **RedisKeyValueStateIterator.java** (Caso localizzato): 1 smell, 72 LOC. Ideale per testare l'efficacia dell'IA su interventi mirati.

Strategia di Refactoring (Incremental Prompting)

Per ogni classe sono state generate **quattro varianti (C1, C2, C3, C4)** fornendo a Microsoft Copilot input incrementali per guidare il refactoring:

- **C1:** Solo report statico di SonarCloud (nessun test fornito).
- **C2:** Report statico + suite di test **Black-Box** (Category Partition).
- **C3:** Input di + informazioni sul **Control-Flow** (Instruction/Branch Coverage).
- **C4:** Input di + risultati del **Mutation Testing**.

Strategia di Testing per la Validazione

La validazione del refactoring si è basata su test manuali progettati con la tecnica **Category Partition**, **Controlflow**, **MutationTest** accompagnati da test automatici definiti con: **Randoop**, **EvoSuite**, **LLM**.

In conclusione, questa validazione è stata indispensabile per trasformare un semplice esperimento di generazione di codice in un'analisi scientifica sulla **neutralità semantica** e sull'**utilità pratica** dell'IA nel processo di manutenzione del software.

3. Risultati

3.1 Risultati Milestone 1

I risultati della prima fase hanno portato alla creazione di un dataset evolutivo composto da **9.864 istanze**.

Statistiche sulla Difettosità

- **Istanze totali:** 9.864.
- **Istanze Buggy (Actual YES):** 713, corrispondenti a circa il **7,2%** del totale.
- **Istanze Sane (Actual NO):** 9.151.

Questo sbilanciamento (classe minoritaria dei bug < 10%) conferma la necessità delle tecniche di bilanciamento (SMOTE) applicate nelle fasi successive.

Efficacia del Processo di Labeling

L'analisi è partita da un bacino di **1.193 ticket JIRA** validi (Bug, Closed/Resolved, Fixed).

Il processo di **linkage Jira-Git** ha mostrato un'altissima efficacia, con il **94,89%** dei ticket (1.132) collegati correttamente ai commit di riferimento.

L'applicazione dell'algoritmo **SZZ** a livello di classe ha permesso di utilizzare direttamente **711 ticket** per etichettare le classi difettose, garantendo una tracciabilità precisa tra difetto segnalato e codice sorgente.

Qualità del Dato e Stabilità

Per garantire la pulizia del dataset, è stata effettuata una selezione rigorosa delle release. Per il progetto Storm, sono state **rimosse 24 release su 36** per mitigare il fenomeno dello **Snoring**.

Questa scelta metodologica ha permesso di abbattere il tasso di "bug dormienti", assicurando che le classi etichettate come "sane" non contengano in realtà difetti scoperti solo in versioni molto successive alla chiusura del dataset.

In sintesi, i risultati della Milestone 1 hanno prodotto un dataset maturo, caratterizzato da un labeling accurato (SZZ + Proportion total) e protetto dai bias temporali tipici dei progetti software longevi.

Caratterizzazione del debito tecnico

- **Prevalenza degli Smell:** Su un totale di 9.864 istanze, ben **6.436 (circa il 65,2%)** presentano almeno un code smell (`code_smells > 0`), mentre solo 3.428 (34,8%) risultano "pulite".
- **Correlazione con la Difettosità:** È interessante notare che la stragrande maggioranza delle classi buggy appartiene al gruppo delle classi con smell. Delle 713 classi buggy totali, ben **597 (l'83,7%)** sono anche classi "smelly". Questo dato giustifica empiricamente l'interesse nell'analizzare il debito tecnico come possibile indicatore di rischio.

3.2 Risultati Milestone 2

L'analisi della Milestone 2 si è focalizzata sull'identificazione del miglior modello predittivo per individuare le classi buggy nel progetto Apache Storm. Per massimizzare l'efficacia della predizione, sono stati messi a confronto **6 diversi predittori**, derivanti dalla combinazione di tre algoritmi di Machine Learning (**RandomForest, NaiveBayes, IBk**) testati in due diverse configurazioni:

- **Senza Feature Selection:** utilizzando l'intero set di metriche estratte (21 feature).
- **Con Feature Selection (Approccio Filter):** utilizzando il sottoinsieme ottimizzato di feature selezionate tramite CfsSubsetEval e BestFirst.

Identificazione del Miglior Classificatore

Dalla comparazione sistematica, **RandomForest senza Feature Selection** è risultato il miglior classificatore in assoluto [conversazione]. Il modello ha dimostrato una superiore capacità discriminativa e una maggiore stabilità statistica rispetto ai modelli lazy (IBk) o probabilistici (NaiveBayes).

La scelta di eleggere questo modello come **BClassifier** (oracolo per le fasi successive) è supportata dai seguenti risultati emersi dalla valutazione cross-validation:

- **Solidità della Kappa Statistic:** Il modello ha ottenuto valori di Kappa superiori a **0.60**, indicando un accordo molto forte con i dati reali, ben lontano dalla classificazione casuale e nettamente superiore

alla baseline **ZeroR** (che pur avendo un'accuratezza nominale del 93%, otteneva Kappa e Recall pari a zero).

- **Capacità Discriminativa (AUC):** L'area sotto la curva ROC si è attestata su valori estremamente elevati (**0.958**), confermando l'ottima capacità del modello di distinguere tra classi sane e difettose indipendentemente dalla soglia decisionale.
- **Utilità Pratica (Precision/Recall):** Il modello garantisce un ottimo bilanciamento, raggiungendo una **Precision del 0.806%**. Questo significa che circa 3 segnalazioni su 4 del modello sono corrette, consentendo agli sviluppatori di ottimizzare lo sforzo di ispezione del codice.

Di seguito viene riportata la tabella comparativa che sintetizza le prestazioni dei classificatori analizzati nelle configurazioni principali: **Tabella 1: metriche dei classificatori**

3.3 Risultati Milestone 3

I risultati ottenuti tramite la **What-if Analysis** hanno permesso di quantificare l'impatto teorico dei code smell sulla difettosità del software seguendo un approccio controfattuale.

In linea con quanto concordato, i risultati principali si riferiscono alla configurazione con il miglior predittore (**RandomForest senza Feature Selection**) addestrato sull'intero Dataset A.

Impatto della Rimozione degli Smell

Dall'analisi sistematica dei quattro dataset sono emersi i seguenti valori, sintetizzati nella **Tabella 2: descrizione e risultati dei dataset sintetici**

Il valore fondamentale cercato dalla Milestone 3 è la stima delle **classi buggy potenzialmente prevenute** ($\Delta Buggy$), calcolata come differenza tra le predizioni nel set reale "smelly" (B+) e quelle nello scenario a debito zero (B):

- **Classi potenzialmente prevenute: 22.**
- **Riduzione della difettosità in B+: 4,02%.**
- **Riduzione della difettosità sull'intero progetto (A): 3,11%.**

Discussione dei Risultati

L'analisi evidenzia che, sebbene l'eliminazione totale dei code smell porti a un miglioramento della qualità stimata, l'effetto è contenuto (circa il 4% delle classi a rischio). Questo suggerisce che in Apache Storm gli smell non sono il driver principale della difettosità, la quale risulta probabilmente dominata da altri fattori come l'instabilità del codice (Churn) o la complessità dimensionale (LOC).

3.4 Risultati Milestone 4

L'ultima fase dello studio ha valutato se il refactoring automatizzato (IA-driven) fosse in grado di azzerare i code smell mantenendo l'integrità del sistema. I risultati mostrano un netto contrasto tra la capacità dell'IA di agire su classi localizzate rispetto a classi complesse.

Risultati per il Caso Complesso: StatsUtil.java

Per questa "God Class", l'IA ha mostrato un'ottima capacità di abbattimento degli smell statici, ma con differenze qualitative tra le versioni e una costante criticità sulla compilabilità, **Tabella 3: risultati del refactoring della classe StatsUtil.java**

Nonostante l'abbattimento massiccio del numero di smell (fino all'83% di riduzione in C4), la classe StatsUtil.java non è mai risultata compilabile in nessuna delle quattro varianti generate.

Miglioramenti Statici: L'IA ha applicato correttamente pattern di modernizzazione come l'aggiunta del costruttore privato per la classe utility, l'estrazione di literal duplicati (costanti **EXEC_ID** e **COMP_ID**), la rimozione di cast ridondanti e l'uso del diamond operator e di computelfAbsent.

Cause del Fallimento: La natura di "God Class" di StatsUtil (1700+ LOC e 62 metodi) ha reso impossibile per l'LLM mantenere la coerenza globale dei tipi e delle dipendenze Thrift in un unico passaggio. Poiché le classi non compilavano, non è stato possibile calcolare le metriche dinamiche (Coverage e Mutation Score), rendendo la validazione puramente statica.

Rischio Elevato: La variante C4, pur essendo la "più pulita" con soli 44 smell residui, è risultata la più rischiosa in quanto ha modificato arbitrariamente la firma del metodo privato thrifyCompPageData, alterando l'integrità strutturale rispetto alla baseline.

Risultati per il Caso Localizzato: RedisKeyValueStateIterator.java

In questo scenario di refactoring mirato, l'IA ha ottenuto un successo pieno e uniforme su tutte le varianti. **Tabella 4: risultati del refactoring della classe RedisKeyValueStateIterator.java**

In netta contrapposizione, il refactoring della classe RedisKeyValueStateIterator.java è stato un successo totale.

Efficacia Uniforme: Tutte le varianti (c1-c4) hanno azzerato l'unico smell presente già al primo step di prompting.

Stabilità e Validazione: Ogni versione è risultata pienamente compilabile e ha superato tutti i test di regressione (progettati con Mockito per simulare l'interazione con Redis), dimostrando la massima affidabilità dell'IA su componenti di piccole dimensioni.

Sintesi dei Risultati

Il confronto tra tutte le varianti evidenzia che:

- **Saturazione del miglioramento:** Nelle classi semplici, l'incremento di informazioni nei prompt non produce variazioni significative se lo smell è già stato rimosso correttamente.
- **Trade-off tra Smell e Rischio:** Nelle classi complesse come StatsUtil, la variante con meno smell (c4) è paradossalmente quella a più alto rischio di introduzione di bug sintattici o strutturali a causa delle modifiche più invasive alle firme dei metodi.
- **Limiti dell'Automazione:** L'analisi di tutte le versioni conferma che l'IA eccelle nella pulizia del codice locale, ma fatica a mantenere la coerenza globale necessaria per la compilazione in classi di grandi dimensioni.

4. Minacce alla validità e scelte soggettive

In questa sezione vengono discussi i limiti tecnici riscontrati e le decisioni arbitrarie prese durante le varie fasi del progetto.

Minacce alla Validità

- **Mapping Release-Commit e Branch Multipli:** La struttura del repository di Apache Storm presenta molteplici branch di sviluppo attivi contemporaneamente, il che ha reso difficile identificare l'esatto istante di chiusura di alcune release. Una restrizione rigorosa avrebbe richiesto la selezione di una sola porzione della release, ma questa scelta, sommata alle tecniche di pulizia dei dati, avrebbe ridotto drasticamente le dimensioni del dataset finale. Si è quindi deciso di **accettare un'imprecisione potenziale su metriche temporali (come l'Age)**, poiché un'ispezione ha rivelato che il problema interessava **meno del 4% delle classi** nel dataset.
- **Incompatibilità di EvoSuite:** Durante la Milestone 4, il tentativo di utilizzare **EvoSuite** per la generazione automatica dei test è fallito a causa di un'incompatibilità tecnica insormontabile tra lo strumento e la versione Java utilizzata nella release di Storm analizzata. Il problema è stato riscontrato sia tentando l'esecuzione tramite plugin Maven, sia in modalità standalone, rendendo necessario l'affidamento esclusivo a test manuali e automatici (Randoop e LLM) per la validazione.

Scelte Soggettive

Oltre alle limitazioni tecniche, sono state effettuate scelte metodologiche specifiche basate sull'analisi dei dati:

- **Filtraggio delle Classi Semplici:** Per la fase di refactoring, sono state deliberatamente escluse tutte le classi con **meno di 70 LOC e meno di 5 metodi**. Questa scelta è stata fondamentale per evitare scenari di refactoring banali (come l'aggiunta di un costruttore privato in una utility class di 30 righe), garantendo un terreno di prova significativo per valutare le capacità di Microsoft Copilot su logiche complesse.
- **Parametro P nel metodo Proportion (Media):** Nonostante la presenza di outlier, si è scelto di utilizzare la **media del parametro P** anziché la mediana per la stima della IV nei ticket incompleti. Questa decisione riflette la volontà di non ignorare i bug con cicli di vita più lunghi, fornendo una visione più conservativa e completa della storia dei difetti.
- **Configurazione di SMOTE al 100%:** Per gestire lo sbilanciamento delle classi (7% buggy), è stato impostato il parametro **SMOTE a 100**. Questa configurazione raddoppia esattamente il numero di istanze della classe minoritaria, creando un bilanciamento sufficiente per istruire il **RandomForest** senza saturare il dataset con troppi dati sintetici che potrebbero causare overfitting.
- **Mappatura della Fixed Version (FV):** In presenza di più fix version su Jira, è stata selezionata la più vecchia, assumendo che rappresenti il primo momento in cui il difetto è stato risolto ufficialmente.

5. Conclusioni

I principali risultati emersi dalle quattro milestone sono i seguenti:

- **Efficacia della Defect Prediction:** Il modello **RandomForest** (configurato con SMOTE al 100% e senza feature selection) si è confermato lo strumento più robusto per identificare le classi bug-prone, ottenendo un'**AUC superiore a 0.94**.
- **Impatto dei Code Smell:** La What-if Analysis ha quantificato l'influenza del debito tecnico sulla difettosità. Sebbene l'azzeramento degli smell porti a una riduzione stimata della difettosità (circa **22 classi buggy prevenute**, pari al **4.02%** nel set delle classi smelly), i risultati suggeriscono che in Apache Storm gli smell non sono il driver primario dei bug, i quali risultano più influenzati dalla complessità strutturale (LOC) e dall'instabilità del codice (Churn).
- **Potenziale e Limiti dell'IA nel Refactoring:** L'esperimento con **Microsoft Copilot** ha rivelato un netto dualismo. L'IA si è dimostrata **estremamente efficace nel refactoring localizzato** (classe `RedisKeyValueStateIterator`), azzerando il debito tecnico senza compromettere la stabilità funzionale. Tuttavia, nel caso della "God Class" `StatsUtil.java`, l'IA ha mostrato difficoltà critiche nel mantenere la coerenza globale: nonostante l'abbattimento dell'83% degli smell statici, **nessuna variante è risultata compilabile**, evidenziando il rischio di "allucinazioni sintattiche" su componenti di grandi dimensioni.

In conclusione, il lavoro conferma che la rimozione degli smell è una pratica necessaria per migliorare la manutenibilità e ridurre il rischio di difetti, ma non deve essere considerata una soluzione universale. L'integrazione di strumenti di **IA generativa** nel processo di manutenzione rappresenta un'opportunità straordinaria per interventi mirati e correzioni sintattiche, ma richiede ancora una **supervisione umana intensiva** per garantire l'integrità strutturale e semantica del sistema, specialmente in architetture software complesse e stratificate come quelle di Apache Storm.

Appendice C

Metriche di dimensione, complessità e instabilità

- **LOC (Lines of Code):** misura la dimensione fisica della classe; classi più grandi tendono a essere più difficili da comprendere, testare e mantenere.
- **Churn:** somma delle linee aggiunte, rimosse e modificate; rappresenta il livello di instabilità evolutiva della classe.
- **LOC Added:** quantifica la crescita funzionale introdotta nelle revisioni.
- **Max LOC Added:** individua i commit con il maggiore incremento di codice, spesso più difficili da validare correttamente.
- **Max Churn:** evidenzia le revisioni più invasive, potenzialmente destabilizzanti.
- **Average LOC Added (total):** media delle linee aggiunte lungo la storia evolutiva, utile per distinguere crescita progressiva e picchi anomali.
- **Number of Revisions (NR):** numero di revisioni subite dalla classe; valori elevati possono indicare instabilità o requisiti volatili.
- **Change Set Size:** numero di file modificati insieme alla classe, usato come indicatore di accoppiamento e rischio di regressioni.

Metriche storiche e fattore umano

- **NFix:** numero di bug fix effettuati sulla classe nell'intervallo corrente; classi già difettose in passato tendono a concentrarvi ulteriori difetti.
- **NAuth:** numero di autori distinti che hanno modificato la classe; una maggiore frammentazione della responsabilità può aumentare il rischio di incoerenze.

Metriche temporali

- **AGE:** età della classe, espressa in settimane; distingue codice giovane da codice legacy.
- **Weighted AGE:** età pesata sulla dimensione della classe, per rappresentare più realisticamente il costo di mantenimento accumulato.

Metriche di commit aggregate per classe

- **NS (Number of Modified Subsystems):** numero di sottosistemi coinvolti insieme alla classe nei commit; più domini architetturali implicano più coordinamento e più rischio.
- **Entropy:** misura quanto le modifiche siano distribuite tra file e componenti; un'alta entropia indica cambiamenti più frammentati e difficili da tracciare.
- **FIX:** variabile booleana che indica se la classe è stata coinvolta in un'attività di bug fixing.
- **NDEV:** numero di sviluppatori coinvolti nei commit recenti; riflette la complessità collaborativa del periodo considerato.

Metriche cumulative

- **NFix_Total:** numero totale di bug fix subiti dalla classe lungo tutta la sua storia.
- **Number_Revisions_Total:** numero complessivo di revisioni subite dalla classe.

- **Average Change Set_Total:** media storica della dimensione dei change set.
- **Average Churn_Total:** churn medio cumulativo della classe.
- **NAuth_Total:** numero totale di sviluppatori distinti che hanno lavorato sulla classe dalla sua introduzione.

DATASET	FEATURE SELECTION	CLASSIFIER	BALANCING	IR RECALL	IR PRECISION	AUC	KAPPA
1	YES	IBK	NO	0.299	0.314	0.681	0.253
2	YES	IBK	YES	0.348	0.306	0.69	0.269
3	YES	NAIVEBAYES	NO	0.881	0.098	0.713	0.054
4	YES	NAIVEBAYES	YES	0.904	0.093	0.711	0.043
5	YES	RANDOMFOREST	NO	0.486	0.831	0.948	0.59
6	YES	RANDOMFOREST	YES	0.53	0.749	0.942	0.595
7	NO	IBK	NO	0.343	0.355	0.668	0.298
8	NO	IBK	YES	0.381	0.331	0.68	0.299
9	NO	NAIVEBAYES	NO	0.953	0.083	0.743	0.023
10	NO	NAIVEBAYES	YES	0.956	0.082	0.746	0.02
11	NO	RANDOMFOREST	NO	0.523	0.857	0.962	0.627
12	NO	RANDOMFOREST	YES	0.577	0.806	0.958	0.649

Tabella 1: metriche dei classificatori

DATASET	DESCRIZIONE	ISTANZE	ACTUAL BUGGY(A)	ESTIMATED BUGGY(E)
DATASET A	Dataset completo originale	9.864	713	708
DATASET B+	Classi con Smell	6.436	-597	547
DATASET B	Scenario controfattuale	6.436	-	525
DATASET C	Classi già sane	3.428	138	194

Tabella 2: descrizione e risultati dei dataset sintetici

VARIANTE	INPUT PROMPT	NSMELL	DELTA SMELL
C0	Classe originale	265	
C1	C0+ report sonarcloud	47	-218
C2	Prompt(c1)+ testBB	47	-218
C3	Prompt(c2)+ ControlflowTest	49	-216

C4	Prompt(c3)+ mutationtest	44	-221
-----------	-----------------------------	----	------

Tabella 3: risultati del refactoring della classe StatsUtil.java

VARIANTE	INPUT PROMPT	NSMELL	DELTA SMELL
C0	Classe originale	1	
C1	C0+ report sonarcloud	0	-1
C2	Prompt(c1)+ testBB	0	-1
C3	Prompt(c2)+ ControlflowTest	0	-1
C4	Prompt(c3)+ mutationtest		-1

Tabella 4: risultati del refactoring della classe RediskeyValueStateIterator.java